Java Arrays and Methods: Category C Solutions

STEP SEM-3 · CodInClub | Powered by BridgeLabz

Problem 1: Hackathon Score Curve Booster

Level: Easy

Approach

• The method receives a reference to the caller's array, so adding the bonus to each element changes the original array directly.
• The return type is **void**; nothing is reassigned in main().
• The result is printed with `Arrays.toString(scores)` as required.

Solution

```
// ---------- Problem 1 ----------
static void curveScores(int[] scores, int bonus) {
    for (int i = 0; i < scores.length; i++) {
        scores[i] += bonus;
    }
}

// main
int[] scores = {70, 85, 60};
curveScores(scores, 10);
System.out.println(Arrays.toString(scores));
```

Output (verified by compiling and running)

```
[80, 95, 70]
```

Matches the expected `[80, 95, 70]`.

Problem 2: Duplicate Team Name Finder

Level: Easy

Approach

• Plain nested loops, no Collections classes.
• The inner loop starts at `i + 1`, so every pair is compared only once and the first duplicate in scan order is reported.
• Names are compared with `equals()`, which is case-sensitive as the constraints require.

Solution

```
// ---------- Problem 2 ----------
static String findDuplicateTeam(String[] teamNames) {
    for (int i = 0; i < teamNames.length; i++) {
        for (int j = i + 1; j < teamNames.length; j++) {
            if (teamNames[i].equals(teamNames[j])) {
                return "Duplicate Found: " + teamNames[i];
            }
        }
    }
    return "No Duplicates Found";
}
```

Output (verified by compiling and running)

```
Duplicate Found: ByteForce
No Duplicates Found
```

Both examples shown above (a duplicate case and a no-duplicate case) match the expected output.

Problem 3: Top-3 Podium Finder

Level: Intermediate

Approach

• Single left-to-right pass with three running variables: first, second, third. No sorting and no Arrays.sort.
• A new score that is greater than or equal to *first* pushes first into second and second into third. Using **>=** lets a tied top score occupy both first and second, which gives [90, 90, 82] for the example.
• Otherwise it is compared against second, then third, shifting only what is needed.
• Time O(n), extra space O(1).

Solution

```
// ---------- Problem 3 ----------
static int[] findTopThreeScores(int[] scores) {
    int first = Integer.MIN_VALUE;
    int second = Integer.MIN_VALUE;
    int third = Integer.MIN_VALUE;
    for (int s : scores) {
        if (s >= first) {
            third = second;
            second = first;
            first = s;
        } else if (s >= second) {
            third = second;
            second = s;
        } else if (s > third) {
            third = s;
        }
    }
    return new int[] { first, second, third };
}
```

Output (verified by compiling and running)

```
[90, 90, 82]
```

Matches the expected `[90, 90, 82]`.

Problem 4: Hackathon Seating Grid Optimizer

Level: Intermediate

Approach

• `rowAverage(int[] row)` is a private helper that only computes and returns one row's average.
• `classifyRows` calls it exactly once per row and is the only place that decides Quiet vs Buzzing.
• Works for jagged grids because each row's own length is used, and an empty row is guarded against division by zero.
• Below the threshold means Quiet Zone; at or above it means Buzzing Zone.

Solution

```
// ---------- Problem 4 ----------
private static double rowAverage(int[] row) {
    if (row.length == 0) return 0.0;
    int sum = 0;
    for (int v : row) sum += v;
    return (double) sum / row.length;
}

static String classifyRows(int[][] seatingScores, int threshold) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < seatingScores.length; i++) {
        double avg = rowAverage(seatingScores[i]);
        String zone = avg < threshold ? "Quiet Zone" : "Buzzing Zone";
        if (i > 0) sb.append(" | ");
        sb.append("Row ").append(i).append(": ").append(zone);
    }
    return sb.toString();
}
```

Output (verified by compiling and running)

```
Row 0: Quiet Zone | Row 1: Buzzing Zone | Row 2: Quiet Zone
```

Matches the expected `Row 0: Quiet Zone | Row 1: Buzzing Zone | Row 2: Quiet Zone`.

Problem 5: Placement Drive Shortlisting and Ranking Engine

Level: Advanced

Approach

• **Thresholds (my choice, as the problem allows):** CGPA-only filter passes at CGPA >= 7.0; the combined filter passes at CGPA >= 6.5 *and* coding score >= 60.
• **Composite score:** 10 × CGPA + 0.5 × coding score. This reproduces the sample values (Aisha 102.0, Rohit 100.5, Karan 85.0).
• Candidate implements `Comparable<Candidate>` with a descending comparison on the composite score, so a plain `Arrays.sort(shortlisted)` does the ranking.
• Fields are private and final (encapsulation); eligibility checks are overloaded static methods.
• Meena (CGPA 6.0) fails both checks despite her high coding score, so she is excluded.

Solution

```
// ---------- Problem 5 ----------
static class Candidate implements Comparable<Candidate> {
    private final String name;
    private final double cgpa;
    private final int codingScore;

    public Candidate(String name, double cgpa, int codingScore) {
        this.name = name;
        this.cgpa = cgpa;
        this.codingScore = codingScore;
    }

    public String getName() { return name; }
    public double getCgpa() { return cgpa; }
    public int getCodingScore() { return codingScore; }

    // composite = 10 * CGPA + 0.5 * codingScore
    public double compositeScore() {
        return cgpa * 10 + codingScore * 0.5;
    }

    @Override
    public int compareTo(Candidate other) {
        // descending by composite score
        return Double.compare(other.compositeScore(), this.compositeScore());
    }
}

static boolean isEligible(double cgpa) {
    return cgpa >= 7.0;
}

static boolean isEligible(double cgpa, int codingScore) {
    return cgpa >= 6.5 && codingScore >= 60;
}

static String shortlistAndRank(Candidate[] candidates) {
    Candidate[] temp = new Candidate[candidates.length];
    int count = 0;
    for (Candidate c : candidates) {
        if (isEligible(c.getCgpa()) || isEligible(c.getCgpa(), c.getCodingScore())) {
            temp[count++] = c;
        }
    }
    Candidate[] shortlisted = Arrays.copyOf(temp, count);
    Arrays.sort(shortlisted);

    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < shortlisted.length; i++) {
        if (i > 0) sb.append(" | ");
        sb.append(i + 1).append(". ").append(shortlisted[i].getName())
          .append(" (").append(shortlisted[i].compositeScore()).append(")");
    }
    return sb.toString();
}
```

Output (verified by compiling and running)

```
1. Aisha (102.0) | 2. Rohit (100.5) | 3. Karan (85.0)
```

Matches the expected `1. Aisha (102.0) | 2. Rohit (100.5) | 3. Karan (85.0)`.

Complete Source: Solutions.java

Compile and run with `javac Solutions.java && java Solutions`.

```
import java.util.Arrays;

public class Solutions {

    // ---------- Problem 1 ----------
    static void curveScores(int[] scores, int bonus) {
        for (int i = 0; i < scores.length; i++) {
            scores[i] += bonus;
        }
    }

    // ---------- Problem 2 ----------
    static String findDuplicateTeam(String[] teamNames) {
        for (int i = 0; i < teamNames.length; i++) {
            for (int j = i + 1; j < teamNames.length; j++) {
                if (teamNames[i].equals(teamNames[j])) {
                    return "Duplicate Found: " + teamNames[i];
                }
            }
        }
        return "No Duplicates Found";
    }

    // ---------- Problem 3 ----------
    static int[] findTopThreeScores(int[] scores) {
        int first = Integer.MIN_VALUE;
        int second = Integer.MIN_VALUE;
        int third = Integer.MIN_VALUE;
        for (int s : scores) {
            if (s >= first) {
                third = second;
                second = first;
                first = s;
            } else if (s >= second) {
                third = second;
                second = s;
            } else if (s > third) {
                third = s;
            }
        }
        return new int[] { first, second, third };
    }

    // ---------- Problem 4 ----------
    private static double rowAverage(int[] row) {
        if (row.length == 0) return 0.0;
        int sum = 0;
        for (int v : row) sum += v;
        return (double) sum / row.length;
    }

    static String classifyRows(int[][] seatingScores, int threshold) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < seatingScores.length; i++) {
            double avg = rowAverage(seatingScores[i]);
            String zone = avg < threshold ? "Quiet Zone" : "Buzzing Zone";
            if (i > 0) sb.append(" | ");
            sb.append("Row ").append(i).append(": ").append(zone);
        }
        return sb.toString();
```

```
    }

    // ---------- Problem 5 ----------
    static class Candidate implements Comparable<Candidate> {
        private final String name;
        private final double cgpa;
        private final int codingScore;

        public Candidate(String name, double cgpa, int codingScore) {
            this.name = name;
            this.cgpa = cgpa;
            this.codingScore = codingScore;
        }

        public String getName() { return name; }
        public double getCgpa() { return cgpa; }
        public int getCodingScore() { return codingScore; }

        // composite = 10 * CGPA + 0.5 * codingScore
        public double compositeScore() {
            return cgpa * 10 + codingScore * 0.5;
        }

        @Override
        public int compareTo(Candidate other) {
            // descending by composite score
            return Double.compare(other.compositeScore(), this.compositeScore());
        }
    }

    static boolean isEligible(double cgpa) {
        return cgpa >= 7.0;
    }

    static boolean isEligible(double cgpa, int codingScore) {
        return cgpa >= 6.5 && codingScore >= 60;
    }

    static String shortlistAndRank(Candidate[] candidates) {
        Candidate[] temp = new Candidate[candidates.length];
        int count = 0;
        for (Candidate c : candidates) {
            if (isEligible(c.getCgpa()) || isEligible(c.getCgpa(), c.getCodingScore())) {
                temp[count++] = c;
            }
        }
        Candidate[] shortlisted = Arrays.copyOf(temp, count);
        Arrays.sort(shortlisted);

        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < shortlisted.length; i++) {
            if (i > 0) sb.append(" | ");
            sb.append(i + 1).append(". ").append(shortlisted[i].getName())
              .append(" (").append(shortlisted[i].compositeScore()).append(")");
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        System.out.println("--- Problem 1 ---");
```

```
        int[] scores = {70, 85, 60};
        curveScores(scores, 10);
        System.out.println(Arrays.toString(scores));

        System.out.println("--- Problem 2 ---");
        System.out.println(findDuplicateTeam(new String[]{"ByteForce", "CodeCrafters", "ByteForce"}));
        System.out.println(findDuplicateTeam(new String[]{"ByteForce", "CodeCrafters", "NullPointers"}));

        System.out.println("--- Problem 3 ---");
        System.out.println(Arrays.toString(findTopThreeScores(new int[]{45, 82, 79, 90, 33, 90, 61})));

        System.out.println("--- Problem 4 ---");
        int[][] grid = { {40, 50, 45}, {85, 90, 95}, {30, 20, 25} };
        System.out.println(classifyRows(grid, 60));

        System.out.println("--- Problem 5 ---");
        System.out.println(shortlistAndRank(new Candidate[]{
            new Candidate("Aisha", 8.2, 40),
            new Candidate("Rohit", 6.8, 65),
            new Candidate("Meena", 6.0, 90),
            new Candidate("Karan", 7.5, 20)
        }));
    }
}
```